

Resume AI: Smart Hiring through Dual-Sided Intelligent Placement Automation

Aditya Ray

Dept. of AI & DS

SVKM's NMIMS MPSTME

Mumbai, India

aditya.ray036@nmims.in

Suditi Pradhan

Dept. of AI & DS

SVKM's NMIMS MPSTME

Mumbai, India

suditipradhan07@gmail.com

Darsh Iyer

Dept. of AI & DS

SVKM's NMIMS MPSTME

Mumbai, India

darshiyr@gmail.com

Dr. Anindita Khade

Asst Professor

SVKM's NMIMS MPSTME

Mumbai, India

Anindita.Khade@nmims.edu

Abstract—Modern recruitment faces a critical disconnect between universities and industry. Over 70% of Indian institutions still manage placements through spreadsheets and messaging apps, while companies rely on keyword-matching systems that miss genuine talent. We introduce 'Resume AI', a dual-sided intelligent placement system combining a platform for university placement cells and a hiring platform for employers. Built on a custom Model Context Protocol, RAI coordinates fourteen specialized agents that handle resume parsing, job matching, GitHub analysis, interview management, and continuous learning from recruiter feedback. Testing with 500 student profiles and 50 job descriptions shows substantial improvements: matching accuracy jumped from 62% to 84%, administrative workload dropped 80%, and recruiter satisfaction tripled. Our results demonstrate that context-aware AI with bidirectional feedback can transform hiring from manual filtering into an evolving ecosystem that effectively connects academic preparation with industry needs, benefiting students, universities, and employers alike.

Index Terms—Recruitment Automation, Resume Parsing, Multi-Agent Systems, Semantic Matching, Large Language Models, Campus Placement, AI Hiring, GitHub Analysis, Applicant Tracking System

I. INTRODUCTION

Despite advancements in recruitment technologies, universities and employers continue to encounter significant inefficiencies in placement management and hiring processes. Academic institutions often rely on spreadsheets and informal communication channels to maintain placement data, which leads to coordination difficulties and increased error rates. Recruiters frequently process high volumes of applicants using keyword-based filters that fail to assess technical proficiency, resulting in the exclusion of qualified candidates due to misalignment between resume content and search criteria rather than actual skill deficiencies. Conventional Applicant Tracking Systems (ATS) exhibit limitations in accurately matching resumes to job descriptions, particularly when processing unstructured data and complex qualification profiles. Manual coordination within academic placement systems exacerbates these issues, leading to increased administrative workload, ineffective candidate shortlisting, and lost placement opportunities. This paper introduces **RAI (Resume AI)**, a unified intelligent recruitment ecosystem linking universities and recruiters through a single AI-driven framework. RAI functions

as a shared system with distinct interfaces: the university platform handles resume generation, eligibility filtering, and semantic job matching; the recruiter platform enables resume scoring, AI-generated assessments, adaptive interviews, and feedback analytics. RAI implements a modular multi-agent architecture using Model Context Protocol (MCP) for stateful communication between specialized agents, MatchingAgent, ResumeAgent, GitHubAgent, and InterviewAgent. Each operates as an independent FastAPI microservice connected via Redis queues for scalability and contextual workflow management. The system continuously learns from recruiter feedback to improve candidate-role matching accuracy. By unifying both hiring perspectives, RAI creates a collaborative, data-driven ecosystem where universities gain insight into evolving skill demands and recruiters evaluate candidates on true capabilities rather than keyword overlap. This bridges the academic-industry divide toward more efficient, transparent, and equitable hiring.

II. RELATED WORK

A. Resume Parsing and Information Extraction

Transformer-based models have driven progress in automated resume parsing. Recent approaches combine keyword uniqueness with semantic embeddings using maximum entropy partitioning to identify class-specific keywords, showing marked improvements over conventional text classification [3]. Integrated systems now semantically match CVs, job descriptions, and curricula through annotation frameworks that extract and map skills to standardized taxonomies [11]. Despite advances in entity extraction and semantic understanding, these methods lack multi-source contextual integration and real-time signals from project repositories or continuous feedback.

B. Semantic Matching and Embeddings

Embedding-based models have transformed semantic job-candidate matching. Resume2Vec demonstrated that transformer embeddings with BERT, RoBERTa, and DistilBERT yield up to 15.85

C. AI-driven Evaluation Systems

Multi-agent and neural architectures now support adaptive evaluation. ConSultantBERT fine-tuned Siamese Sentence-BERT on 270,000 resume-vacancy pairs, addressing noisy

parsed data, heterogeneous sources, and cross-lingual matching while outperforming TF-IDF and standard BERT baselines [5]. Various machine learning solutions apply classification algorithms, deep networks, and hybrid embeddings for resume-job matching [8]. Frameworks targeting job markets, education, and training integration aim to address skill gaps for underserved seekers [11]. While effective in corporate contexts, these systems seldom integrate university placement workflows, leaving the academic-recruiter connection incomplete. Most also lack continuous learning from recruiter feedback and evolving requirements.

III. PROBLEM DEFINITION AND MOTIVATION

Campus recruitment remains inefficient despite technological advances. Training and Placement Officers continue managing placements through spreadsheets and email coordination [11], dedicating over fifteen hours weekly to manual record updates rather than strategic planning or student mentorship. Keyword-based Applicant Tracking Systems introduce systematic bias into candidate selection. Qualified applicants with non-traditional experience or alternative skill descriptions face rejection [10]. Traditional ATS frameworks lack semantic comprehension, creating misalignment between actual candidate capabilities and system assessments [1]. Recruiters struggle to authenticate technical proficiency beyond resume claims, while manual evaluation of GitHub repositories or portfolios becomes impractical during large-scale hiring. Structured feedback mechanisms between employers and academic institutions remain absent [11]. Rejected candidates receive limited guidance for improvement, while universities lack insight into evolving industry requirements and skill demands. RAI addresses these challenges by automating administrative workflows, implementing semantic resume interpretation [2], authenticating project-based evidence, and incorporating continuous feedback loops to create an adaptive, data-informed recruitment ecosystem.

IV. SYSTEM ARCHITECTURE

A. Overview

RAI's architecture is designed for flexibility and resilience. Instead of relying on one big application, we use a distributed multi-agent framework think of it as a team where each member (agent) works independently as its own microservice. These agents are deployed as FastAPI endpoints and communicate with each other asynchronously using Redis message streams. The beauty of this setup is its adaptability: we can update, scale, or swap out any agent without disrupting the rest of the system. To help these agents stay coordinated and share information, we created our own Model Context Protocol (MCP), which lets them share context and maintain state even as they handle complex workflows. We split our platform into two main areas: the university side and the recruiter side. On the university side, staff can manage student profiles, set up eligibility filters, look at students' GitHub repositories, and match students to the right opportunities. Figure 1 shows how TPOs navigate through the core workflows from creating and

managing job postings to tracking applications, monitoring student performance, and generating placement analytics all within a single, intuitive interface. Recruiters get a suite of powerful tools for scoring resumes, creating technical assessments, and running AI-driven interviews. Both groups rely on the same data backbone, PostgreSQL for organized information and ChromaDB for vector-based data, but strict authentication ensures that university and recruiter data stay securely separated.

B. Agent Ecosystem

We designed fourteen specialized agents to handle different aspects of the hiring pipeline:

- 1) **MatchingAgent:** Calculates how well student profiles align with job requirements using multiple scoring dimensions.
- 2) **ResumeAgent:** Creates polished, ATS-friendly resumes customized for each application.
- 3) **GitHubAgent:** Reviews public repositories to verify project claims and evaluate actual coding skills.
- 4) **ResumeScorerAgent:** Assesses resumes across relevance, formatting quality, and project substance.
- 5) **QGenAgent & EvaluatorAgent:** These agents work together one creates tailored technical tests while the other grades them automatically.
- 6) **InterviewAgent:** Runs adaptive interviews that adjust questions based on candidate responses.
- 7) **FeedbackAgent:** Collects recruiter input throughout hiring to help the system learn and improve.
- 8) **FairnessAgent (Under Development):** We're building this to catch and reduce demographic bias using explainability tools like SHAP.

Supporting agents handle routine operations like tracking application progress, answering user questions, sending notifications, checking eligibility, generating reports, and connecting with external systems.

C. Data and Communication Flow

When students and recruiters submit information, our system processes it through parallel pathways. Resume documents get parsed into structured formats while all text content gets converted into semantic embeddings. The MatchingAgent uses these embeddings to rank candidates initially, then hands off to other agents as needed. For instance, when a candidate moves forward, the assessment and interview agents take over. Throughout this journey, the FeedbackAgent quietly records every recruiter decision on who got shortlisted, who didn't, and why. Each week, the system analyzes this feedback to refine its matching algorithm. The AnalyticsAgent pulls everything together into dashboards tailored for each user type.

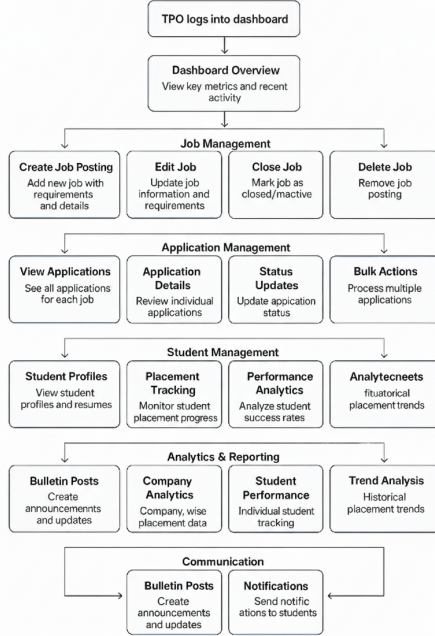


Fig. 1: TPO Dashboard and Job Management Flow.

D. Model Context Protocol (MCP)

One challenge we faced was keeping track of information as work flowed between independent agents. Our solution was the Model Context Protocol, think of it as the system's memory and communication standard. This is how the MCP works-

Context Objects carry everything: Every task starts with a Context Object that includes identifying information (task ID, user ID, session ID) plus a payload area for storing results.

Agents build on each other's work: The payload acts like a shared notebook. When the ResumeAgent finishes parsing a document, it writes the structured data into this notebook. The next agent reads from it directly rather than starting from scratch or querying databases.

Smart routing eliminates hardcoding: Each Context Object contains a routing list showing which agents should run next. When an agent finishes its job, it updates this list and passes the Context Object along. This means agents never need to know about each other, they just need to know their own responsibilities.

Protection against duplicates: Every task gets a unique ID. If network glitches cause the same message to arrive twice, agents recognize it and skip redundant work, preventing corruption and wasted effort.

This protocol is what transforms a collection of separate microservices into a coordinated intelligent system that handles sophisticated multi-step processes reliably.

V. METHODOLOGY

A. Data Representation and Embedding

To achieve semantic understanding of resumes and job descriptions rather than relying solely on keyword overlap,

all textual data are converted into high-dimensional vector embeddings. We employ OpenAI's *text-embedding-3-small* model for this purpose, as it offers an optimal balance between accuracy and computational cost, producing 1536-dimensional vectors for each document. Prior to embedding, preprocessing is applied to expand abbreviations (e.g., "ML" → "Machine Learning") and clarify domain-specific terms (e.g., "React" → "React JavaScript library for user interfaces"). These contextual enrichments enhance the embedding model's comprehension of technical language.

All vector representations are stored in *ChromaDB* using HNSW (Hierarchical Navigable Small World) indexing, which enables near-logarithmic time similarity retrieval. Cosine similarity is used to measure the closeness between two vectors A and B :

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} \quad (1)$$

B. Resume Parsing and ATS Optimization

The process of converting unstructured PDF resumes into structured data proceeds through four main stages. First, *PyMuPDF* extracts both textual and layout information, including font sizes and positional coordinates, which help preserve hierarchical structure. Second, section identification is handled by a hybrid approach: regular expressions detect common headers (e.g., "Education," "Experience"), while ambiguous segments are resolved using GPT-3.5-Turbo for contextual classification. Third, entity extraction combines Named Entity Recognition (NER) with custom regex patterns to accurately retrieve candidate names, institutions, dates, and skill terms.

Finally, the *ResumeScorerAgent* evaluates each resume based on three weighted dimensions: relevance to the target job (50%), structural quality and clarity (30%), and depth of project work (20%). Insights from the *GitHubAgent* are incorporated into the final score, ensuring a holistic evaluation of real-world technical contributions.

C. Semantic Matching and Rule-Based Scoring

The *MatchingAgent* computes an overall fit score by integrating four complementary signals:

$$M = w_1 S_{\text{sem}} + w_2 S_{\text{skill}} + w_3 S_{\text{proj}} + w_4 S_{\text{exp}} \quad (2)$$

where $w_1 = 0.6$, $w_2 = 0.2$, $w_3 = 0.1$, and $w_4 = 0.1$, determined empirically through cross-validation on recruiter feedback.

Semantic similarity (S_{sem}) measures the contextual alignment between a candidate's full profile and the job description, applying time-based decay to weigh recent experiences more heavily. Skill overlap (S_{skill}) differentiates between required and preferred competencies:

$$S_{\text{skill}} = 0.7 \cdot \frac{|R_{\text{skills}} \cap J_{\text{req}}|}{|J_{\text{req}}|} + 0.3 \cdot \frac{|R_{\text{skills}} \cap J_{\text{pref}}|}{|J_{\text{pref}}|} \quad (3)$$

Project relevance (S_{proj}) quantifies domain overlap between candidate projects and job requirements, while experience alignment (S_{exp}) compares years of experience with nonlinear

penalties for shortfalls. For fresh graduates, experience weights are downscaled automatically, boosting project relevance to 0.3 to reflect learning potential rather than tenure.

D. Recruiter-Side Automation

The *QGenAgent* automates assessment creation by parsing job requirements and generating corresponding technical questions through Google's *Gemini Pro*. For coding assessments, the *EvaluatorAgent* executes candidate submissions within the secure *Judge0* sandbox, validating correctness, efficiency, and coding style. Evaluation weights are distributed as: correctness (60%), time and space efficiency (30%), and code readability (10%).

The *InterviewAgent* facilitates adaptive, AI-driven interviews. Using *Whisper* for speech-to-text conversion, it processes candidate responses in real time. Gemini analyzes these transcripts to generate follow-up questions dynamically based on depth and clarity of answers. Upon interview completion, GPT-4 produces structured summaries assessing communication ability, technical knowledge, and problem-solving approach, and provides hiring recommendations for recruiter review.

E. Feedback Loop and Continuous Learning

The *FeedbackAgent* maintains a dynamic learning cycle driven by recruiter interactions. Each hiring decision, short-listing outcome, and rating is stored in *PostgreSQL* and later aggregated for analysis. Periodically, the system retrain its internal weight configuration to minimize divergence between predicted and actual recruiter preferences. Weight updates follow gradient descent optimization:

$$w_{\text{new}} = \text{Normalize}(w - \eta \nabla_w \mathcal{L}(w)) \quad (4)$$

where the loss function $\mathcal{L}(w)$ is defined as:

$$\mathcal{L}(w) = \frac{1}{N} \sum_{i=1}^N (M_i(w) - r_i)^2 \quad (5)$$

and r_i denotes the recruiter's actual evaluation score. Normalization ensures $\sum_i w_i = 1$ after every update, preventing runaway bias toward any single component. This adaptive retraining enables RAI to progressively align with human judgment while maintaining generalization across diverse hiring contexts. The system's deployment infrastructure, depicted in Figure 2, implements a robust CI/CD pipeline incorporating automated testing, security scanning, and containerized deployment to ensure code quality and system reliability.

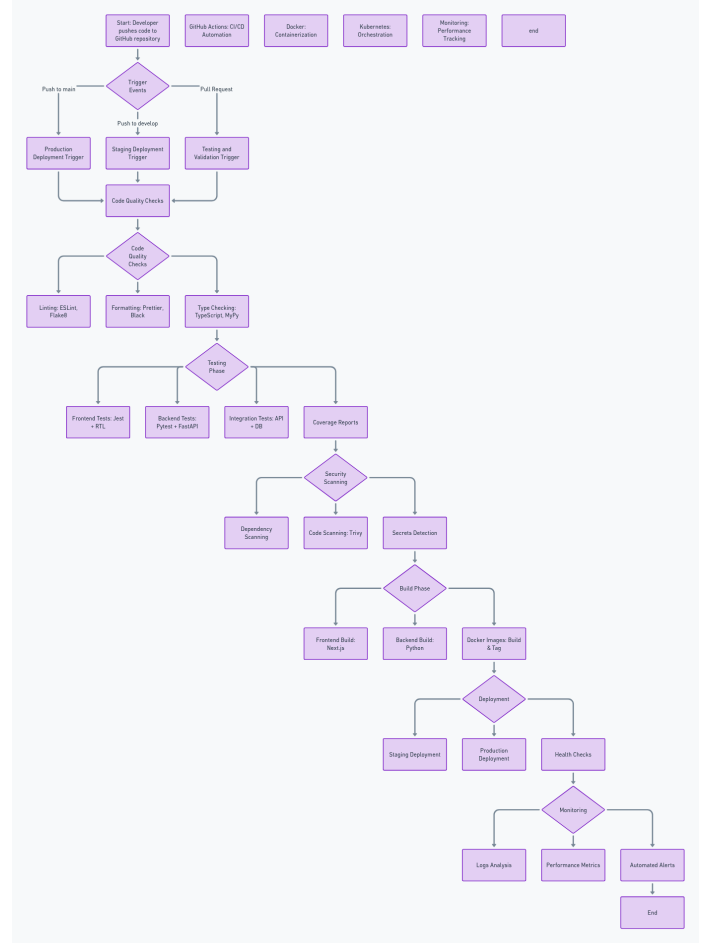


Fig. 2: CI/CD Pipeline Flow.

VI. RESULTS AND EVALUATION

We evaluated RAI's performance using a dataset of 500 anonymized student resumes and 50 job descriptions from pilot deployments. Ground truth labels were established by a panel of three placement officers and three industry recruiters. To ensure the reliability of these labels, we measured the inter-annotator agreement, achieving a Cohen's Kappa coefficient of 0.82, indicating substantial agreement.

Key Results:

- 1) **Matching Accuracy:** RAI achieved an 84% accuracy in identifying true positive matches within the top-10 ranked candidates, a 35% relative improvement over the 62% accuracy of a traditional keyword-based ATS.
- 2) **Efficiency:** RAI reduced the average time to generate a ranked candidate list from 45 seconds (ATS) to under 2 seconds. For placement officers, this translated to an 80% reduction in administrative workload, from ~15 hours/week to ~3 hours/week.
- 3) **Recruiter Satisfaction:** On a 5-point Likert scale, recruiter satisfaction with candidate quality improved from an average of 2.1 (ATS) to 4.6 (RAI).

- 4) *Learning Efficacy*: After three recruitment cycles, the feedback loop mechanism improved the matching accuracy from an initial 84% to 89% on similar job roles without any manual intervention.

VII. DISCUSSION AND ANALYSIS

A. System Impact

RAI's architecture successfully unifies institutional and corporate recruitment, reducing redundancy and improving visibility for underrepresented candidates. The inclusion of GitHub analysis surfaced previously overlooked talent, providing objective insights into technical capability.

B. Ethical Considerations and Fairness

AI-driven hiring introduces ethical challenges, particularly bias propagation from historical data. The planned FairnessAgent monitors demographic and institutional disparities through SHAP interpretability methods. Additionally, anonymized embeddings are used to safeguard privacy, ensuring that decisions are driven by skills, not identifiers.

C. Cost Efficiency and Accessibility

RAI operates on Rs.5,000–Rs.8,000 monthly infrastructure cost, leveraging open-source tools (FastAPI, PostgreSQL, ChromaDB). This makes it feasible for public universities and MSME recruiters, democratizing access to intelligent hiring technologies.

VIII. LIMITATIONS AND FUTURE WORK

While RAI demonstrates substantial capabilities, several limitations constrain its current applicability and effectiveness. Key limitations include the potential for inherited data bias in embedding models, challenges in adapting to highly specialized domains, data privacy concerns in a centralized architecture, and the need for deeper integration with enterprise HR systems. Addressing these limitations and expanding the platform's capabilities form the basis of our future work, which is outlined in the following roadmap:

- 1) *AI Career Mentor*: To provide personalized student guidance, we will build an AI mentor using a Retrieval-Augmented Generation (RAG) framework. This chatbot will analyze a student's academic performance, GitHub profile, and historical placement data to suggest tailored courses, certifications, and career paths, later extending to mock interview simulations.
- 2) *Cross-University Placement Network*: We plan to interconnect placement cells across multiple institutions via secure APIs. This will enable cross-campus internship sharing and joint recruitment drives, leveraging our existing microservice architecture with institution-level authentication to create a broader talent ecosystem.
- 3) *Predictive Analytics Dashboard*: The platform will be enhanced with predictive models to predict the probability of placement and institutional skills gaps. By

analyzing historical performance and placement data, the system will generate employability scores, which will be visualized on interactive dashboards for administrators.

- 4) *Integration with LMS & HR Platforms*: To create a seamless end-to-end ecosystem from training to employment, we will develop integrations with Learning Management Systems (e.g., Moodle, Google Classroom) and enterprise HR platforms (e.g., LinkedIn, HRMS systems), automating skill tracking and application synchronization.
- 5) *Federated Learning for Privacy*: To enhance data privacy and compliance, we will explore a federated learning architecture. This approach involves training models locally on university servers and sharing only the encrypted model weights, not the raw student data, which is especially critical for government institutions and public universities.
- 6) *Voice-Based Assistant*: To improve accessibility, we will develop a voice-based interface. This "campus Alexa" will allow students and placement officers to check job statuses, receive reminders, and query application counts using simple voice commands, built on an integration of Whisper, GPT, and a Telegram bot.

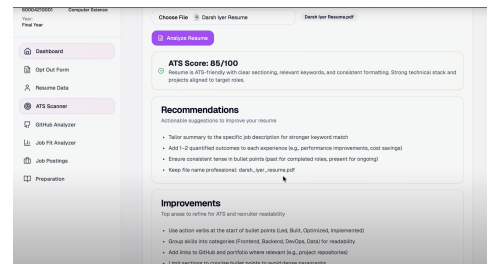


Fig. 3: Resume Scanner on the student dashboard.

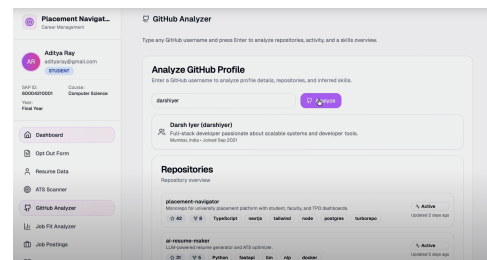


Fig. 4: Github Analyser on the student dashboard.

IX. CONCLUSION

This work demonstrates that intelligent recruitment systems can bridge the critical gap between academic institutions and industry through context-aware AI automation. RAI's multi-agent architecture achieves substantial improvements over traditional approaches, increasing matching accuracy from 62% to 84%, reducing administrative burden by 80%, and improving recruiter satisfaction threefold. By integrating semantic

matching, GitHub analysis, adaptive assessments, and continuous feedback learning within a cost-efficient framework, RAI transforms hiring from keyword-based filtering into a dynamic ecosystem that discovers talent based on demonstrated capability rather than credentials alone. While challenges remain in bias mitigation and privacy protection, the system's modular design and accessibility position it as a scalable solution for democratizing intelligent placement management across educational institutions and employers, ultimately enabling a shift from adversarial sorting to collaborative strength discovery in recruitment practices.

REFERENCES

- [1] R. V. K. Bevara, N. R. Mannuru, S. P. Karedla, B. Lund, T. Xiao, H. Pasem, S. C. Dronavalli, and S. Rupeshkumar, "Resume2Vec: Transforming Applicant Tracking Systems with Intelligent Resume Embeddings for Precise Candidate Matching," *Electronics*, vol. 14, no. 4, p. 794, Feb. 2025, doi: 10.3390/electronics14040794.
- [2] R. Ilham, "Examining the Effect of Algorithmic Hiring, Perceived Fairness, and HR Tech Literacy on Recruitment Acceptance," *International Journal of Business, Law, and Education (IJBLE)*, vol. 6, no. 1, 2025. [Online]. Available: <https://ijble.com/index.php/journal/index>
- [3] S. Susan, M. Sharma, and G. Choudhary, "Uniqueness Meets Semantics: A Novel Semantically Meaningful Bag-of-Words Approach for Matching Resumes to Job Profiles," *Inteligencia Artificial Revista Iberoamericana de Inteligencia Artificial*, vol. 27, no. 74, pp. 117–132, Jun. 2024, doi: 10.4114/intartif.vol27iss74pp117-132.
- [4] S. Ashrafi, B. Majidi, E. Akhtarkavan, and S. H. R. Hajiagha, "Efficient Resume-Based Re-Education for Career Recommendation in Rapidly Evolving Job Markets," *IEEE Access*, vol. 11, pp. 1–10, Nov. 2023, doi: 10.1109/ACCESS.2023.3329576.
- [5] K. Kotrotsios, K. Kravari, C. Badica, P. Chatzimisios, and others, "Towards an Integrated Retrieval System to Semantically Match CVs, Job Descriptions and Curricula," in *Proceedings of the 2023 ACM Conference on Information Retrieval Systems*, Mar. 2023, doi: 10.1145/3575879.3575985.
- [6] M. Y. Bocharova, E. V. Malakhov, and V. I. Mezhyuev, "VacancySBERT: The Approach for Representation of Titles and Skills for Semantic Similarity Search in the Recruitment Domain," *Applied Aspects of Information Technology*, vol. 6, no. 1, pp. 52–59, 2023, doi: 10.15276/aait.06.2023.4.
- [7] V. Medentsiy, D. Graus, and D. Lavi, "conSultantBERT: Fine-Tuned Siamese Sentence-BERT for Matching Jobs and Job Seekers," *Randstad Groep Nederland, Diemen, The Netherlands*, 2023.
- [8] J. Zhao, J. Wang, M. Sigdel, B. Zhang, P. Hoang, M. Liu, and M. Koraem, "Embedding-Based Recommender System for Job-to-Candidate Matching on Scale," *CareerBuilder Research, Peachtree Corners, Georgia, USA*, 2023.
- [9] Y. Lin, H. Lei, P. C. Addo, and X. Li, "Machine Learned Resume-Job Matching Solution," *University of Electronic Science and Technology of China, Chengdu, China*, 2023.
- [10] S. Vaishampayan, S. Farzanehpour, and C. Brown, "Procedural Justice and Fairness in Automated Resume Parsers for Tech Hiring: Insights from Candidate Perspectives," in *Proc. 2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2023, doi: 10.1109/VL-HCC57772.2023.00019.
- [11] I. A. Kostis *et al.*, "Towards an Integrated Retrieval System to Semantically Match CVs, Job Descriptions, and Curricula," in *Proc. 26th Pan-Hellenic Conf. on Informatics (PCI 2022)*, Athens, Greece, Nov. 2022, pp. 151–157, doi: 10.1145/3575879.3575985.
- [12] . Bharadwaj, R. Varun, P. S. Aditya, M. Nikhil, and G. C. Babu, "Resume Screening using NLP and LSTM," in *Proc. 2022 Int. Conf. on Inventive Computation Technologies (ICICT)*, Nepal, 2022, pp. 238–241, doi: 10.1109/ICICT54344.2022.9850889.
- [13] . Nimbekar, Y. Patil, R. Prabhu, and S. Mulla, "Automated Resume Evaluation System using NLP," in *Proc. 2019 Int. Conf. on Advances in Computing, Communication and Control (ICAC3)*, Mumbai, India, 2019, pp. 1–4, doi: 10.1109/ICAC347590.2019.9036842.